

Don't hard code the device ID with 0, because zero is not always the ID of the first adapter. In case a system has two adapters and the first adapter (ID 0) is disabled, then the first available adapter is the one with ID 1.

inquiry_info structure is used to store the Bluetooth device type address (see Figure 8).

The first field of this structure, *bdaddr_t*, gives the address of the detected Bluetooth device. The *dev_class* field gives information about the type of detected device like whether it is a phone, printer or computer.

```
inquiry_info *ii = NULL;
```

You can get more information about *dev_class* from <http://www.wensley.org.uk/c/inq/reporter.c>

Getting Bluetooth addresses and the types of Bluetooth devices

After selecting the local Bluetooth adapter and file handle to the host controller interface, it's time to scan for Bluetooth devices nearby. *hci_inquiry* discovers Bluetooth devices and stores basic information about them, like Bluetooth addresses, type of device, etc, in the *inquiry_info* structure.

```
int hci_inquiry ( int dev_id , int time_len , int max_rsp ,
const uint8_t *lap , inquiry_info **ii , long flags );
```

Enquiry or discovery of devices lasts for at the most 1.28 * *time_len* seconds and, at the most, *max_rsp* devices will be returned in the output parameter *ii*.



Note: To get enough time for enquiry, use 8 for *time_len*, that is 10.24 seconds, and 255 for *max_rsp*. If you want to flush the cache of previously detected devices, set flags to *IREQ_CACHE_FLUSH*; otherwise, use 0 and *hci_inquiry* will also return those Bluetooth devices that are out of range, but still in cache.

Now we have the Bluetooth addresses of all available devices. With the help of these addresses, you can get the names of the devices.

```
typedef struct {
    bdaddr_t          bdaddr;
    uint8_t            pscan_rep_mode;
    uint8_t            pscan_period_mode;
    uint8_t            pscan_mode;
    uint8_t            dev_class[3];
    uint16_t           clock_offset;
}__attribute__((packed)) inquiry_info;
```

Figure 8: Structure *inquiry_info*

```
--S ./a.out
[ Net Capture Obex Phone ] Cell Phone 00:6B:5B:A4:6C:1E Micromax x101
[ Net Render Capture Audio Phone ] Misc Computer 9C:2A:70:DA:A0:56 dell-desktop-0
```

Figure 9: Output of searched Bluetooth devices

```
int hci_read_remote_name ( int fh , const bdaddr_t *ba , int
len , char *name , int timeout );
```

hci_read_remote_name tries timeout milliseconds to use the *fh* file handle for querying the user-friendly name of the device with the help of the Bluetooth address *ba*, and store len byte of friendly name into *name*.

After this, do not forget to free the *inquiry_info* structure and close the file handle to the host controller interface.

```
free ( ii );
close ( fh );
```

How to run the program and get an output

First, compile this program by invoking *gcc* and linking with *libbluetooth*:

```
# gcc scan_bt_devices.c -lbluetooth
```

See Figure 9 for the output. Though it need not be mentioned, before running this program, the Bluetooth of the device must be ON. You can switch ON your Bluetooth manually or you can switch it ON by using the *rfkill* command:

```
rfkill unblock bluetooth
```

You can switch it OFF similarly:

```
rfkill block bluetooth
```

RFCOMM and L2CAP sockets

L2CAP connections are like UDP protocol and both are packet based protocols, but there is a difference between them. In UDP, it is possible that the two packets, sent from a computer, may reach randomly, which means that the second packet may reach first. But in L2CAP, the first packet will always reach first.

RFCOMM connections are like TCP. But RFCOMM supports only 30 unreserved channels (known as ports in TCP) and no reserved channels in a single machine, while TCP supports 65,535 ports.

In L2CAP, ports are known as protocol service multiplexers (PSM) and take odd-numbered values between 1 to 32,767, where odd-numbered PSMs from 1 to 4,095 are reserved while odd-numbered PSMs from 4,097 to 32,765 are unreserved.

L2CAP connections are encapsulated within ACL (asynchronous connection oriented logic) connections and RFCOMM connections are transported within L2CAP connections, so RFCOMM connections are also encapsulated within ACL connections.